
Durable Context

How LLM assistants consume what you tell them,
and what reliable skill execution actually requires

Adam Young — Model Citizen

September 2026

A technical white paper.
Vendor documentation surveyed and cited as of 9 September 2026.

Contents

Abstract	1
1. Everyone is rebuilding the same foundation every morning	2
2. Four surfaces people mistake for one	2
3. What the vendors actually document	3
4. The window got bigger. Your odds did not.	7
5. The same question does not get the same answer	8
6. What determinism you can actually buy	9
7. Context as infrastructure, not input	10
8. A reference model	11
9. How you would know it worked	12
10. What this evidence does not support	13
11. A five-minute test	13
References	14
Disclosure	17
Appendix A — Vendor comparison, gaps shown as gaps	17

Abstract

Every mainstream LLM assistant now offers some way to make context durable: memory that fills itself, projects that hold instructions, knowledge files, connectors, custom agents. Users reasonably assume these surfaces behave like configuration — set once, applied consistently. This paper examines the published documentation for five widely deployed services (Anthropic Claude, OpenAI ChatGPT, Google Gemini, Microsoft 365 Copilot, and Perplexity) and finds that assumption unsupported.

Two findings carry the argument. First, only one of the five documents what happens to a user's context when a conversation exceeds the model's window; the other four document it nowhere, leaving users unable to know whether their earlier instructions were summarized, truncated, or dropped. Second, across all five vendors combined, exactly three statements of instruction precedence exist. On four of five services, a user cannot determine from documentation which of their own durable surfaces wins when two of them disagree.

Those gaps sit on top of a research literature that is unambiguous about the underlying substrate. Effective context is materially shorter than advertised context. Identical prompts at temperature zero do not produce identical outputs, for reasons that have little to do with the prompt and much to do with how the server batches and numerically executes the request. Semantically neutral changes to prompt formatting have been measured to move accuracy by as much as 76 points, and paraphrasing an instruction reorders which model scores best.

The conclusion is not that these systems are unusable. It is that reliability cannot be obtained from inside the chat window, because the chat window is not where the durable part lives. Context has to be treated as infrastructure — written down once, versioned, owned, and portable across the tools that consume it. This paper sets out what the evidence supports, what it does not, a four-part taxonomy of the context surfaces users actually have, a vendor-neutral reference model for governing them, and a set of measurements by which an organization could tell whether any of it worked.

1. Everyone is rebuilding the same foundation every morning

Watch anyone competent use an AI assistant for real work and you will see the same sequence. They open a new chat. They spend the first two hundred words explaining who they are, what team they sit on, what the deliverable is for, what tone their leadership expects, which three systems the data comes from, and what went wrong the last time. Then they ask the question.

Tomorrow they will do it again.

This gets diagnosed as a skill problem, and a large training industry has grown up around that diagnosis. Learn to prompt better. Be more specific. Give it examples. All of that advice is true and none of it addresses what is actually happening, which is that the person is hand-carrying the foundation to the job site every single morning because there is nowhere to leave it.

The instinct is to treat context as input — something you supply along with the request, sized to the request, discarded after. That instinct is wrong in a specific and expensive way. Context is not input. It is the surface the work stands on, and like any load-bearing surface it is supposed to be poured once, inspected, and left in place. When it is not, everything built on it is a one-off, and every one-off has to be rebuilt from scratch by whoever needs it next.

The systems have partly noticed this. Every vendor examined here now ships something that looks like durable context — memory, projects, custom instructions, knowledge files, saved agents. The trouble is that these surfaces are documented as features and not as infrastructure. Infrastructure comes with specifications: capacity, failure behavior, precedence, what happens under load. Features come with a settings toggle and a help article.

This paper reads the specifications that exist, names the ones that do not, and works out what a person or an organization would have to do to get reliable behavior anyway.

A note on scope. This is not an argument that AI assistants are unreliable and should be avoided. It is an argument about where the reliability has to come from. Anyone who has spent time with governed data models will recognize the shape of it immediately: the trust problem is never in the surface you are looking at. It is three layers down, in the thing nobody wrote out.

2. Four surfaces people mistake for one

Before anything can be measured, the thing being measured needs naming. Users experience “the AI remembering things” as a single capability. It is four, they fail differently, and conflating them is the reason most durability advice does not survive contact with a real workflow.

Turn context is what you typed, plus whatever the interface attached to it — files, the visible conversation, the current page. It is complete, it is exactly what you supplied, and it is gone when the conversation ends or when the window fills. This is the only surface most users consciously manage, and it is the least durable of the four.

Durable instruction is the surface that behaves most like configuration: profile or custom instructions, project instructions, Gems, custom GPTs, declarative agents, Skills. You write it, it persists, and it applies to some defined scope. This is where most of the value is, and it is also where the documentation is thinnest.

Memory is what the service inferred and kept without being asked. It is automatic on all five services studied, it is account-scoped by default on all five, and it is the only one of the four surfaces that changes when you are not looking.

Retrieval is what gets fetched and ranked at answer time — knowledge files, connected systems, search results, enterprise indexes. It is not stored context at all; it is a query executed on your behalf, whose results you generally do not see and whose ranking you cannot inspect.

Two of these are deterministic in principle. Turn context is exactly what you sent. Durable instruction is exactly what you wrote. Two are not: memory is a model-maintained representation of you, and retrieval is a similarity search. The mistake almost everyone makes is putting load-bearing content on the two probabilistic surfaces because they are easier to fill, and then being surprised when the behavior drifts.

Hold onto that split. It structures the rest of the paper.

3. What the vendors actually document

The survey below covers Anthropic Claude, OpenAI ChatGPT, Google Gemini, Microsoft 365 Copilot, and Perplexity. Every claim is drawn from vendor documentation current as of September 2026 and is cited. Where a vendor does not document something, that is recorded as an absence rather than filled with inference — because for a paper about durability, the absences are the finding.

3.1 Nine of twenty context-window figures are not published

Service	Free	Mid paid	Top consumer	Enterprise
Claude	not documented	1M (newest models)	1M	1M
ChatGPT	27K instant / reasoning “varies”	54K / 256K	128K / 400K	not documented
Gemini	32K	128K (AI Plus)	1M (Pro/Ultra)	not documented
M365 Copilot	—	—	—	not documented at all
Perplexity	not documented	not documented	not documented	not documented

Google publishes the cleanest tier disclosure of the five and is the only vendor stating a free-tier figure — a 31× gap between free and Pro [1]. OpenAI’s numbers appear only on its pricing page, mapped to two labels, “GPT Instant” and “GPT Reasoning,” which the company does not connect to its named models anywhere public [2]. They read as product-tier routing buckets rather than model names, but OpenAI does not say so — and either way, a ChatGPT user cannot determine which window a given message received. Anthropic’s figures are model-scoped and clearly stated for paid plans, but the article is titled for paid plans and gives no free-tier number [3]. Microsoft publishes no figure for Microsoft 365 Copilot at all, which is defensible given that Copilot is architected as an orchestrated retrieval system rather than a raw window — but it means the quantity cannot be reasoned about. Perplexity, which multiplexes across ten models, nine of them third-party, publishes nothing; its own help-center URL promising an answer about token limits resolves to a page that does not contain one.

The table matters less than what sits underneath it, which is §4 of this paper: advertised context is not effective context, and the published number would be a weak guide even where it exists.

3.2 One vendor of five says what happens when the window fills

This is the sharpest finding in the survey.

Anthropic documents that Claude does not truncate. It summarizes: “When your conversation approaches the context window limit, Claude summarizes earlier messages to make room for new content” [3]. Two qualifiers on the behavior are documented alongside it. It “only works for users with code execution enabled” — so a user without that setting can still hit a hard limit — and “rare edge cases (such as very large first messages) may still encounter context limits” [4]. The same article says both that automatic summarization “does not count towards your usage limit, and allows conversations to continue indefinitely in most cases” and that “longer conversations that trigger automatic context management consume more of your usage limit”; the two sentences sit in tension and this paper does not resolve them. Separately, and helpfully, the full transcript remains visible in the interface even though the model’s working context no longer holds it.

At the developer level Anthropic documents an API primitive called *compaction*, with a default trigger at 150,000 input tokens and the explicit note that “the API automatically drops all content blocks prior to the compaction block” [5]. Anthropic does not state that *claude.ai* uses this API. The two should be read as two separately documented facts, not one mechanism.

OpenAI, Google, Microsoft, and Perplexity document this nowhere. Extensive searching of each vendor’s help and learning centers returned no statement of whether a conversation exceeding the window is truncated, summarized, or silently dropped.

The asymmetry deserves emphasis, because the two behaviors are not interchangeable. Truncation is lossy but deterministic: the oldest tokens go, and a user who knows the rule can reason about what survived. Summarization is lossy and non-deterministic: what survives is whatever a summarizer judged important, which the user did not choose, cannot inspect, and cannot reproduce. Both are legitimate engineering choices. Only one of the five tells you which one you got.

For anyone relying on instructions given early in a long working session, this is the whole ballgame. You cannot know whether they are still there.

3.3 A project can change its retrieval semantics without being edited

Anthropic documents that project knowledge is held in full context until it approaches the window limit, at which point “Claude will automatically enable RAG mode to expand your project’s capacity” — up to ten times, using a project knowledge search tool that “intelligently searches and retrieves only the most relevant information,” with a visual indicator shown to the user [6][7].

Read that as an engineer rather than as a user and it is a documented transition from deterministic grounding to probabilistic grounding, triggered by data volume rather than by any user action. The same project, with the same instructions, answering the same question, is running a different retrieval architecture on Tuesday than it ran on Monday, because someone added files on Monday night. Anthropic deserves credit for documenting this plainly and for surfacing an indicator; the point is not that Anthropic behaves worse but that it is the only one of the five that tells you.

Microsoft is the other vendor that describes retrieval mechanics in citable terms, and describes them more explicitly than anyone: the semantic index works “by creating vectorized indices,” enabling “fast and accurate similarity search and retrieval of data based on their vector distance or similarity,” with documented query expansion that pulls in nearby words to broaden the search [8]. That is a definitive answer to the deterministic-versus-probabilistic question for Copilot, and it is ranked, not exact.

OpenAI does not state how Projects files or custom GPT knowledge are retrieved. Its guidance to builders is instead to “test your GPT in Preview after uploading files to make sure it uses the content the way you expect” [9] — an instruction that is itself an admission, though not a specification. Google and Perplexity document nothing on the mechanism.

3.4 Three precedence statements exist across five vendors

If durable instruction surfaces are configuration, they need a conflict-resolution rule. Here is every published rule found:

1. OpenAI’s Model Spec defines an authority hierarchy — root, system, developer, user, guideline — and states that lower-level content, “including its own previous messages,” must not influence interpretation of higher-level principles [10].
2. OpenAI states that “Project instructions only apply inside the respective project and will override your global custom instructions” [11].
3. Anthropic states that where organization instructions and individual instructions conflict, “organization instructions take precedence” [12].

That is the complete list.

The first governs message roles, not persistence surfaces — so it tells you developer beats user, and says nothing about where an automatically synthesized memory ranks against an explicitly written custom instruction. That question is unanswered at OpenAI, and it is the central question for anyone relying on written configuration.

Anthropic documents when to use profile instructions versus project instructions versus Skills, and says they “can be used independently or in combination” [54], but publishes no override ordering among them. Google publishes no instruction hierarchy at all. Microsoft publishes none across custom instructions, saved memories, inferred chat-history details, and agent instructions. Perplexity publishes none across project instructions, memories, and search history.

So: on four of five services, a user who writes two durable instructions that quietly disagree has no documented way to predict which one governs. This is the reproducibility problem in its clearest form, and it is evidenced by absence — which is legitimate evidence, provided it is reported as “not documented” rather than as “does not exist.”

3.5 Memory is on by default, and it is not always yours

Memory is automatic on all five services. The governance around it varies more than most users would expect.

Microsoft’s documentation is the most specific and, read together, the most striking. Microsoft documents the feature as in preview and “subject to change,” which should temper any conclusion drawn from it. With that said: “By default, Enhanced personalization is turned on,” and

“no action is required to turn on Copilot memory; you only need to turn off Copilot memory.” Memories are stored in a hidden folder in the user’s Exchange mailbox, inheriting mailbox security and encryption. And then three statements in the same article: “Retention policies and retention labels configured in Purview by organization admins don’t apply to Copilot memory”; “Memory and personalization actions don’t generate audit log entries in Purview”; and, to the direct question of whether a tenant admin can restrict what information is added to memory, “No” [13]. Each of those is Microsoft’s own plain statement. The synthesis is this paper’s: that is a durable, automatically populated, per-user context store that is on by default, exempt from retention policy, not audit-logged, and not restrictable in content.

Perplexity is the ownership outlier. For Enterprise organizations, “memories created within an Enterprise organization belong to the organization, not the individual user,” and departing users lose access to memories created during their tenure [14]. Every other vendor studied treats memory as the user’s.

Anthropic is the portability outlier and the only vendor documenting memory import and export [15]. It is also the only one with per-project memory spaces distinct from account memory, and it defaults memory off for Team and Enterprise while defaulting it on for individual plans [16].

OpenAI’s memory is the most architecturally elaborate: saved memories and automatic memory coexist with a separate chat-history reference channel, and a background synthesis process the company calls “dreaming,” now in its third version, which includes temporal awareness — the documented example being a memory updating from planning a trip to having taken it [17]. Stored memory is therefore not an append-only log but a mutable, model-maintained representation. A memory a user verified in March may read differently in September without the user doing anything.

Google’s is the most restrictively scoped, and produces the survey’s oddest structural finding: personal-context memory is explicitly “not available in Gems” [18]. Google’s two durable surfaces are mutually exclusive. A Gem gives persistent instructions without memory; a normal chat gives memory without the Gem’s instructions. No other vendor documents such a partition.

3.6 The one thing that is genuinely standardized

All five vendors document support for the Model Context Protocol. The current specification, version 2026-07-28, standardizes four primitives — tools, prompts, resources, and tasks — and moved to a stateless core with header-based routing and cacheable list results, changes that read as concessions to gateway deployment at enterprise scale [19]. Adoption is substantial: the protocol’s maintainers report nearly half a billion monthly downloads across the Tier 1 SDKs [19].

MCP is the strongest available argument that portable context is achievable rather than aspirational. It is also worth stating its limit precisely. Microsoft’s Dynamics MCP documentation notes plainly that “Claude Desktop isn’t supported” [20], and Microsoft’s declarative agent documentation observes that OpenAI’s Apps SDK “builds on the MCP Apps standard and adds extra functionality specific to ChatGPT,” with not all capabilities available across both [55]. MCP standardizes the wire protocol. It does not standardize client capability parity, and a paper that claims otherwise will be corrected by the first reader who has tried to move a server between

hosts.

4. The window got bigger. Your odds did not.

The industry's answer to context problems has been to sell more context. Windows have grown from thousands of tokens to a million. The reasonable inference — put everything in, and the model will find what it needs — is contradicted by four independent lines of evidence.

Position is not neutral. Liu et al. held a context fixed and moved only the position of the document containing the answer. Performance follows a U-shape: best when the needed information sits at the beginning or the end, worst in the middle. In some configurations, performance with the relevant document in the middle fell below the same model's performance given no documents at all. Critically, models explicitly trained for longer contexts showed the same curve as their shorter siblings — enlarging the window does not flatten it [21].

The follow-up matters more than the finding. Hsieh et al. showed the cause is an intrinsic U-shaped attention bias: tokens at the start and end receive disproportionate attention regardless of their relevance. Calibrating that positional component out improved retrieval-augmented performance by up to ten percentage points [22]. So this is a bias, not an inability. The model can read the middle. It is disinclined to.

Advertised context is not effective context. Three efforts converge here, and the convergence is what makes the claim robust. RULER extended needle retrieval with multi-hop tracing and aggregation across 17 models: of models claiming 32K context or more, only about half sustain acceptable performance at 32K [23]. NoLiMa removed the lexical shortcut by constructing needles with minimal word overlap with the question, forcing semantic rather than string matching; of 13 models advertising 128K or more, eleven fall below half their own short-context baseline at 32K tokens, with GPT-4o dropping from a 99.3% baseline to 69.7% [24]. Levy, Jacoby and Goldberg held the reasoning task constant and padded the input, isolating length as the only variable, and found degradation at input lengths far below the technical maximum — while also reporting that next-word-prediction quality correlates negatively with long-context reasoning on their data [25].

The benchmark everyone quotes measures the wrong thing. Needle-in-a-haystack testing became the industry's long-context proof point. Anthropic's own report is the cleanest demonstration of its weakness: Claude 2.1 scored 27% on a 200K-token needle retrieval, and adding a single sentence to the prompt raised that to 98% [26]. A 27-to-98 swing from one sentence means the measurement is dominated by prompt formulation, not by model capability. Near-perfect needle scores are close to uninformative.

More relevant context can be worse than less. Chroma's evaluation across 18 models found that a single distractor measurably reduces performance against baseline, four compound it, and the effect amplifies with length. Their most operationally useful result: on a long-memory evaluation, a focused prompt of roughly 300 tokens substantially outperformed the full 113,000-token prompt containing the same relevant information [27]. Chroma sells a retrieval database, and "long context does not replace retrieval" is a commercially convenient conclusion — which is exactly why it should be read alongside RULER, NoLiMa and Levy et al., which are peer-reviewed and have no such interest. The four together are what make the claim hold.

Anthropic's applied team gives the honest framing: treat context as a finite attention budget,

expect “a performance gradient rather than a hard cliff,” and aim for the smallest set of high-signal tokens that gets the outcome [28]. That is a very different design instruction from “paste everything in.”

The practical consequence is uncomfortable for anyone whose durability strategy is a large knowledge folder. Volume is not the objective. Curation is. The evidence supports that much and no more: a focused prompt can beat a much longer one containing the same information, and irrelevant material measurably costs accuracy. It does not license a specific page count.

5. The same question does not get the same answer

Users assume that if they send the same message twice they will get the same reply, and that setting temperature to zero makes this true. Neither holds, and the reason is worth getting exactly right because most published explanations of it are wrong.

Temperature zero is not determinism. Thinking Machines Lab ran 1,000 completions of 1,000 tokens from an identical prompt at greedy decoding on Qwen3-235B. The result was 80 unique completions; the most common one occurred 78 times; the first divergence appeared at token 103 [29].

The common explanation for this is inadequate. The folk account — GPU concurrency plus floating-point non-associativity, therefore atomic adds landing in unpredictable order — does not survive inspection: repeated identical matrix multiplications on a GPU return bitwise-identical results, and the forward pass contains no atomic adds in the relevant path. The actual cause is a lack of *batch invariance*. Production kernels switch reduction strategy depending on the shape of the work they are given, which changes floating-point reduction order. On a shared inference server, batch size varies with other users’ load.

Which yields the sentence that should be quoted whenever someone asks why the answer changed: your output depends on who else was talking to the model at the same moment. You neither control that nor observe it.

It is fixable, at a cost, and this has been replicated. With batch-invariant kernels, the same prompt produced 1,000 identical completions out of 1,000. On a separate throughput benchmark using a smaller model, the deterministic path cost 1.6× to 2.1× the runtime of the default [29]. The SGLang team independently implemented the approach across three attention back-ends and reported 50 trials producing exactly one unique output where the feature was on, against 2 to 18 unique outputs where it was off, at an average 34% slowdown [30]. Separately, a NeurIPS paper establishes that evaluation batch size, GPU count and GPU model materially change generated outputs under limited precision: on AIME’24 with greedy decoding, accuracy standard deviation was 9.15% in BF16 against zero in FP32 [31].

What it costs in practice has been measured. Across 829 code-generation problems, repeated requests to ChatGPT produced no two runs with the same test output on 75.8% of CodeContests problems, 51.0% of APPS problems, and 47.6% of HumanEval problems — with the authors noting explicitly that temperature zero reduces but does not eliminate this [32].

And then the larger problem. Bitwise determinism is the easy half. The hard half is that semantically identical requests do not behave identically either. Sclar et al. varied only formatting — separators, spacing, casing, punctuation — leaving task and semantics untouched, and found performance differences of up to 76 accuracy points on LLaMA-2-13B in few-shot settings, with

sensitivity that does not vanish with model size, more examples, or instruction tuning [33]. Mizrahi et al. paraphrased instructions across 39 tasks, 20 models and 6.5 million instances and found that model *rankings* reorder with phrasing [34]. Example order alone spans near-state-of-the-art to random-guess performance [35]. Multiple-choice option order shifts answers through a token-level prior on the option letters, independent of content [36].

Put the two halves together and the position is this. Determinism of computation is a solved engineering problem with a known price. Robustness to paraphrase is not solved, is much larger in magnitude, and is what people actually mean when they say the assistant is unreliable. Those are different properties, and only the second one is what a user experiences.

The design conclusion follows directly. If the wording of a request materially changes the answer, then the wording of a request cannot be improvised fresh each time by whoever happens to be at the keyboard. It has to be written down, tested, and reused — for the same reason a metric definition gets written into the model rather than retyped into forty different queries.

6. What determinism you can actually buy

Not everything in the previous section is beyond reach. It is worth being precise about which parts are purchasable, because the market is currently selling one of them as though it were the other.

Form is guaranteed. Content is not. Constrained decoding masks the model’s output at each step to only those tokens that can continue a valid string in a target grammar, so an output violating the schema is unreachable rather than merely unlikely [37]. This is a genuine mechanical guarantee, and a cheap one. OpenAI reports its structured-output implementation scoring 100% on an internal complex-schema-following evaluation against under 40% for an earlier model — and documents the limit in the same breath: the model “may still make mistakes within the values of the JSON object” [38]. A published benchmark measures compliance rates above 94% for the strongest implementation while showing meaningful variation between frameworks [39].

So: the shape is deterministic, up to engineering quality. Which valid string gets chosen is governed by everything in §5, unchanged. Conflating these is the most common error in this literature.

Whether structure hurts reasoning is genuinely contested, and should be reported that way. An EMNLP paper reports significant reasoning decline under format restriction [40]. A methodological rebuttal — written by a vendor of a constrained-decoding library, and to be read with that interest in view — shows the compared arms used different prompts and that the paper’s own classification data cut against its headline [41]. A separate benchmark finds constrained decoding roughly three points *better* on reasoning tasks, though its author list overlaps with the framework that performs best in it [39]. A causal analysis across 48 scenarios found no causal effect in 43 of them, and attributed three of the five real effects to the wording of the instruction rather than the output format [42]. The most parsimonious reconciliation comes from a theoretical result: constraint hurts when the grammar leaves no room for intermediate reasoning, and augmenting the grammar to permit unconstrained reasoning between constrained regions recovers up to ten points [43]. The practical form of that is a schema with a free-text reasoning field ahead of the constrained answer fields — not a workaround, but the fix that theory, rebuttal and benchmark all point to.

What is buyable, summarized:

Technique	Guarantees	Cost	Does not address
Batch-invariant kernels	Bitwise-identical output for identical input	1.6–2.1× runtime, or ~34%	Paraphrase sensitivity; retrieval variance
FP32 for greedy decoding	Removes precision-driven variance	Memory and throughput	Everything above
Constrained decoding	Output parses and validates	Low, but framework-dependent	Correctness of values
Written, reused prompts	Removes per-invocation wording variance	Authoring and maintenance	Model-side variance
Curated, minimal context	Reduces distractor and position effects	Curation discipline	Model-side variance

Notice which rows are available to an ordinary user of a consumer assistant. Only the last two. The first three are infrastructure decisions made by whoever runs the inference, and no one using claude.ai or ChatGPT or Copilot is in that position.

That is the crux of this paper. The reliability levers a normal user can actually pull are all on the context side. Which means the context side is where the discipline has to live.

7. Context as infrastructure, not input

Everything above converges on one structural claim: the durable part of an AI workflow cannot live inside the conversation, because the conversation is the least durable surface in the system and the one whose failure behavior is least documented.

So it has to live outside — as written artifacts, owned by the person or the organization, fed into whichever assistant is being used. This is not a novel idea. It is the same argument that governs data modeling, applied to a different asset, and it is worth making the parallel explicit because the failure mode is identical.

An unwritten metric does not stay unwritten and harmless. It forks. It lives half-formed in the WHERE clauses of forty different queries, each one a small private version of what the business means, and the divergence is invisible until two people bring two numbers to the same meeting. An unwritten working method behaves exactly the same way. It lives half-formed across a hundred chat sessions, each one a small private version of how this person works, and the divergence is invisible until the person leaves and takes all hundred of them.

Four things have to be written down. They map onto the four surfaces in §2, and the mapping is the point — each artifact belongs on the deterministic surface, not the probabilistic one.

Identity — who is asking and on whose behalf. Role, scope, the systems you work in, the constraints you operate under, who the audience is. This is what every user retypes in the first two hundred words of every new chat. It belongs in a durable instruction surface, written once, not in memory, where the service will paraphrase it back to you in a form you did not choose.

Decisions — what has been settled and is not up for re-litigation. Naming conventions, the tools chosen, the format leadership expects, the things tried before that did not work. This is the artifact with the highest half-life and the one most often left unwritten, because at the moment of deciding, the decision feels too obvious to record.

Method — how a recurring job is actually done. The steps, the inputs, the checks, the definition of finished. This is where §5 bites hardest: if a request’s phrasing materially changes its answer, a recurring request needs a fixed phrasing that has been tested, not a fresh improvisation each time. A written, tested method is not bureaucracy. It is the defense available to a user who does not control the inference stack — subject to the caveat in §10 that a phrasing tuned on one assistant may not transfer to another.

Evaluation — what good looks like, in checkable terms. The failure modes worth watching for, the things that must be true before output is trusted, the examples of acceptable and unacceptable work. Without this, there is no way to tell whether any of the other three are working, and §9 has nothing to measure.

Three properties make these artifacts infrastructure rather than notes.

They are **written**, meaning a human authored them deliberately, rather than a model having inferred them from conversation. Inferred memory is useful and should be left on; it is not a substitute, because it is a representation the user did not choose and, on at least one service, one that rewrites itself over time [17].

They are **owned**, meaning they exist as files under someone’s control rather than inside a vendor’s account. This is not paranoia about vendors. It is the observation that of five services studied, one documents memory export [15] and one asserts organizational ownership of memories with loss of access on departure [14]. A file survives both.

They are **portable**, meaning they are consumed by whichever assistant is in front of you. The five services in this survey have different windows, different overflow behavior, different precedence, and no common configuration format — but all five accept text, and all five document MCP support [1][11][13][19][49]. Text is the universal floor. MCP standardizes the wire protocol above it, though not, as §3.6 notes, client capability parity.

8. A reference model

The artifacts in §7 need an ownership boundary, because the most common failure in practice is not that nothing is written down — it is that everything is written down in the wrong layer and then lost at the wrong moment.

Three layers, distinguished by what happens to them when the organization changes shape.

Personal. Identity, working preferences, communication style, the accumulated judgment of how this individual does their job well. It travels with the person across teams, employers and tools. It is not company property and should not be stored as though it were.

Team. Role definitions, recurring deliverables, the method by which a repeating job gets done, the local definition of good. This is the layer that survives reorganizations if it exists and evaporates with them if it does not. It is company property, and it is the layer with the highest return, because it is the one that turns one person’s competence into a thing a successor can inherit.

Organizational. Governed skills, approved connectors, data contracts, the definitions that must be identical everywhere. This layer is the direct analogue of the governed semantic model, and it fails the same way when skipped: not with an outage, but with confident divergence.

What breaks when a layer is missing is specific and predictable. Without the personal layer, every individual restarts from zero in every tool, forever. Without the team layer, competence

is unbacked — when someone leaves, their method leaves with them, and the replacement rebuilds it by trial and error. Without the organizational layer, the other two layers quietly disagree with each other at scale, which is worse than having neither, because it is invisible.

One further constraint, drawn from §4: each layer should hold the smallest set of high-signal artifacts that produces the behavior, not everything that could conceivably be relevant. A layer that accumulates without curation risks the distractor effect described in §4, degrading the outcome it was built to improve.

The model above is deliberately vendor-neutral, and the artifacts are files. Any organization can build these three layers with a text editor and a shared drive; nothing in the argument depends on a particular tool, and the disclosure at the end of this paper notes the author's own interest.

9. How you would know it worked

An argument for discipline is worth little if its effects cannot be observed. Four measurements are available, and one common measurement should be avoided.

Use pass^k , not $\text{pass}@k$. The τ -bench work introduced the distinction: $\text{pass}@k$ asks whether the system succeeded at least once in k trials and rises with k ; pass^k asks whether it succeeded on the same task in *all* k trials and falls with k . For anything a person actually depends on, pass^k is the metric that matters. The reported results are sobering — the best agents tested succeeded on under half of tasks at pass^1 , with retail-domain pass^8 falling below roughly 25% [44]. Read them with one caveat the follow-up work supplies: the retail user simulator in that benchmark was itself later measured at a 40% error rate, 12% of it critical, so some of the measured agent inconsistency was simulator inconsistency [52]. Any claim that some architectural change improved reliability should still be met with a request for a pass^k number.

Test paraphrases, not just repeats. Given §5, running the same prompt ten times measures only inference variance. Running ten semantically equivalent phrasings measures the thing that actually breaks. Report the interval, not the best result.

Measure context typed per prompt, and prompts per completed task. These are the two quantities durable context is supposed to reduce, they are directly observable, and they are legible to a non-technical sponsor. If a written role definition is doing its job, the opening two hundred words of setup disappear and the number of clarifying exchanges before a usable answer drops. If neither moves, the artifacts are not being consumed the way their author believes.

Count re-explanations. Over a month, how many times did someone explain the same standing fact to an assistant? This is the crudest measure here and the most persuasive to the person paying for it.

Do not let a model grade itself. LLM-as-judge is a legitimate instrument with documented failure modes, named in the foundational paper alongside its positive result of over 80% agreement with human preferences [53]: position bias that is strongest precisely when two candidates are close in quality [45], verbosity bias, and a self-preference effect established causally rather than correlationally — evaluators score their own generations higher, and strengthening self-recognition strengthens the preference [46]. Randomize presentation order, report position consistency, and never use a model as judge of its own output.

10. What this evidence does not support

A paper arguing for discipline has an obligation to state where the discipline is unproven. Six claims are adjacent to this argument and should not be made.

That bigger context windows are useless. They are not; the degradation is a gradient rather than a cliff [28], and the position component of it is partly correctable at inference time without retraining [22]. The claim supported is that advertised capacity overstates usable capacity, not that long context fails.

That deterministic inference produces reliable systems. It does not. It makes identical input reproduce identical output on one configuration, and says nothing about paraphrase robustness — a much larger source of user-visible variation, and an orthogonal property [33]. These are orthogonal properties and only the second is what users mean.

That structured output improves reasoning. Contested, as set out in §6, and the honest position is that much of the reported effect is a prompt effect in disguise [42].

That multi-agent or sub-agent architectures improve reliability. This is currently unbenchmarked in public. Anthropic advocates the pattern under specific conditions and reports three to ten times the token cost [50]; Cognition argues against it from build experience, with no comparative data [51]; and Anthropic’s own later writing partially converges on that caution. Neither side has published a pass@k figure. Anyone claiming a reliability gain here should be asked for one.

That memory scaffolding reliably helps long-horizon work. One 2026 preprint reports memory scaffolding *reducing* long-horizon performance across all models tested [47]. It is single-source, unreviewed, and insufficient to establish the negative — but it is sufficient to refuse the positive as established.

That prompt formats can be standardized into portable best practice. Format performance correlates only weakly between models [33], so a phrasing tuned on one assistant does not reliably transfer to another. Written method artifacts are worth maintaining; a universal house style for prompt formatting is not supported by the evidence.

One further caveat about the survey itself. Absence of documentation is not evidence of absence of behavior. Where this paper reports that a vendor does not document overflow or precedence, the correct reading is that a user cannot verify it — not that the vendor lacks a rule. Vendor documentation also moves: URLs in this paper’s reference list drifted during research, two help-center articles changed their titles mid-study, and one vendor’s two current pages contradict each other on whether remote MCP is available [48][49]. Anyone re-checking these claims should expect to find some of them superseded, and should date and archive what they cite.

11. A five-minute test

Here is the diagnostic, and it takes about five minutes.

Open a new conversation with whichever assistant you use most. Do not paste anything. Ask it to produce the deliverable you produce most often — the weekly summary, the status note, the analysis, whatever it is — and give it nothing but the request.

Read what comes back and answer three questions. Does it know who you are and who the audience is? Does it use your team’s names for things? Does it match the format you would

have had to correct it into?

If it does know some of it, check where that knowledge came from — an inferred memory you did not write is the §7 failure mode, not a success. If the answer is no, note how many words you would now have to type to fix it. Then multiply that number by how many times you will start a conversation this month.

That number is not a measure of how good you are at prompting. It is the size of the foundation you are carrying to the job site every morning, by hand, because there is nowhere to leave it.

You do not have a prompting problem. You have an infrastructure problem — and unlike the prompting problem, this one you can pour once.

References

Vendor documentation was accessed and verified on 9 September 2026. Vendor help-center URLs drift; dated archival copies are recommended for any downstream citation. The three groupings below are deliberate: several sources frequently cited in this area are vendor or lab publications rather than peer-reviewed work, and are separated here so a reader can weigh them accordingly.

Vendor documentation

[1] Google. “Gemini Apps context window and limits.” support.google.com/gemini/answer/16275805

[2] OpenAI. ChatGPT pricing page. chatgpt.com/pricing

[3] Anthropic. “How large is the context window on paid Claude plans?” support.claude.com/en/articles/86063

[4] Anthropic. “How do usage and length limits work?” support.claude.com/en/articles/11647753

[5] Anthropic. “Compaction.” Claude Platform documentation. platform.claude.com/docs/en/build-with-claude/compaction

[6] Anthropic. “How can I create and manage projects?” support.claude.com/en/articles/9519177

[7] Anthropic. “Retrieval-augmented generation (RAG) for projects.” support.claude.com/en/articles/11473015

[8] Microsoft. “Semantic index for Copilot.” learn.microsoft.com/microsoftsearch/semantic-index-for-copilot

[9] OpenAI. “Knowledge in GPTs.” help.openai.com/en/articles/8843948

[10] OpenAI. “Model Spec,” version 2026-08-18. model-spec.openai.com/2026-08-18.html

[11] OpenAI. “Projects in ChatGPT.” help.openai.com/en/articles/10169521

[12] Anthropic. “Set organization instructions.” support.claude.com/en/articles/14546867

[13] Microsoft. “Microsoft 365 Copilot personalization and memory.” learn.microsoft.com/microsoft-365/copilot/copilot-personalization-memory

[14] Perplexity. “Memory for enterprise organizations.” perplexity.ai/help-center/en/articles/13654357

[15] Anthropic. “Import and export your memory from Claude.” support.claude.com/en/articles/12123587

[16] Anthropic. “Use Claude’s chat search and memory to build on previous context.” support.claude.com/en/articles/11817273

- [17] OpenAI. “Memory and dreaming in ChatGPT.” openai.com/index/chatgpt-memory-dreaming/
- [18] Google. “Personal context in Gemini Apps,” support.google.com/gemini/answer/16598469; and “Create and use Gems,” support.google.com/gemini/answer/15146780
- [19] Model Context Protocol. Specification 2026-07-28 and accompanying release notes. modelcontextprotocol.io/specification/2026-07-28; blog.modelcontextprotocol.io/posts/2026-07-28/
- [20] Microsoft. “Connect agents to Model Context Protocol.” learn.microsoft.com/dynamics365/sales/connect-agents-to-model-context-protocol
- [48] Perplexity. “Local and remote MCPs for Perplexity.” perplexity.ai/help-center/en/articles/11502712
- [49] Perplexity. “Adding custom remote connectors.” perplexity.ai/help-center/en/articles/13915507
- [54] Anthropic. “Understanding Claude’s personalization features.” support.claude.com/en/articles/10185728
- [55] Microsoft. “MCP Apps in declarative agents.” learn.microsoft.com/microsoft-365/copilot/extensibility/plugin-mcp-apps

Vendor and lab publications (not peer-reviewed)

- [26] Anthropic. “Long context prompting for Claude 2.1,” 6 December 2023. claude.com/blog/claude-2-1-prompting
- [27] Hong, K., Troynikov, A., and Huber, J. “Context Rot: How Increasing Input Tokens Impacts LLM Performance.” Chroma Technical Report, 14 July 2025. research.trychroma.com/context-rot
- [28] Rajasekaran, P., Dixon, E., Ryan, C., and Hadfield, J. “Effective context engineering for AI agents.” Anthropic, 29 September 2025. anthropic.com/engineering/effective-context-engineering-for-ai-agents
- [29] He, H., et al. “Defeating Nondeterminism in LLM Inference.” Thinking Machines Lab, 10 September 2025. thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/
- [30] SGLang Team. “Towards Deterministic Inference in SGLang and Reproducible RL Training.” LMSYS Org, 22 September 2025. lmsys.org/blog/2025-09-22-sglang-deterministic/
- [38] OpenAI. “Introducing Structured Outputs in the API,” 6 August 2024. openai.com/index/introducing-structured-outputs-in-the-api/
- [41] Kurt, W. “Say What You Mean: A Response to ‘Let Me Speak Freely’.” [dottxt.ai/engineering/blog](https://dottxt.ai/engineering/blog/say-what-you-mean.html). blog.dottxt.ai/say-what-you-mean.html
- [50] Phillips, C., et al. “Building multi-agent systems: when and how to use them.” Anthropic, 23 January 2026.
- [51] Yan, W. “Don’t Build Multi-Agents.” Cognition, 12 June 2025.

Academic literature

- [21] Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., and Liang, P. “Lost in the Middle: How Language Models Use Long Contexts.” *Transactions of the Association for*

Computational Linguistics, Vol. 12, 2024. aclanthology.org/2024.tacl-1.9/

- [22] Hsieh, C.-Y., Chuang, Y.-S., Li, C.-L., Wang, Z., Le, L., Kumar, A., Glass, J., Ratner, A., Lee, C.-Y., Krishna, R., and Pfister, T. "Found in the Middle: Calibrating Positional Attention Bias Improves Long Context Utilization." *Findings of ACL 2024*. aclanthology.org/2024.findings-acl.890/
- [23] Hsieh, C.-P., Sun, S., Krizan, S., Acharya, S., Rekesh, D., Jia, F., Zhang, Y., and Ginsburg, B. "RULER: What's the Real Context Size of Your Long-Context Language Models?" *COLM 2024*. arxiv.org/abs/2404.06654
- [24] Modarressi, A., Deilamsalehy, H., Dernoncourt, F., Bui, T., Rossi, R. A., Yoon, S., and Schütze, H. "NoLiMa: Long-Context Evaluation Beyond Literal Matching." *ICML 2025*. arxiv.org/abs/2502.05167 — model and threshold counts differ between preprint versions; figures here follow the ICML 2025 version.
- [25] Levy, M., Jacoby, A., and Goldberg, Y. "Same Task, More Tokens: the Impact of Input Length on the Reasoning Performance of Large Language Models." *ACL 2024*. aclanthology.org/2024.acl-long.818/
- [31] Yuan, J., Li, H., Ding, X., Xie, W., Li, Y.-J., Zhao, W., Wan, K., Shi, J., Hu, X., and Liu, Z. "Understanding and Mitigating Numerical Sources of Nondeterminism in LLM Inference." *NeurIPS 2025*. arxiv.org/abs/2506.09501
- [32] Ouyang, S., Zhang, J. M., Harman, M., and Wang, M. "An Empirical Study of the Non-determinism of ChatGPT in Code Generation." *ACM Transactions on Software Engineering and Methodology*. arxiv.org/abs/2308.02828
- [33] Sclar, M., Choi, Y., Tsvetkov, Y., and Suhr, A. "Quantifying Language Models' Sensitivity to Spurious Features in Prompt Design." *ICLR 2024*. arxiv.org/abs/2310.11324
- [34] Mizrahi, M., Kaplan, G., Malkin, D., Dror, R., Shahaf, D., and Stanovsky, G. "State of What Art? A Call for Multi-Prompt LLM Evaluation." *TACL*, Vol. 12, 2024. aclanthology.org/2024.tacl-1.52/
- [35] Lu, Y., Bartolo, M., Moore, A., Riedel, S., and Stenetorp, P. "Fantastically Ordered Prompts and Where to Find Them." *ACL 2022*. aclanthology.org/2022.acl-long.556/
- [36] Zheng, C., Zhou, H., Meng, F., Zhou, J., and Huang, M. "Large Language Models Are Not Robust Multiple Choice Selectors." *ICLR 2024*. arxiv.org/abs/2309.03882
- [37] Willard, B. T., and Louf, R. "Efficient Guided Generation for Large Language Models." [arXiv:2307.09702](https://arxiv.org/abs/2307.09702), 2023.
- [39] Geng, S., Cooper, H., Moskal, M., Jenkins, S., Berman, J., Ranchin, N., West, R., Horvitz, E., and Nori, H. "Generating Structured Outputs from Language Models: Benchmark and Studies." [arXiv:2501.10868](https://arxiv.org/abs/2501.10868), 2025.
- [40] Tam, Z. R., Wu, C.-K., Tsai, Y.-L., Lin, C.-Y., Lee, H.-y., and Chen, Y.-N. "Let Me Speak Freely? A Study On The Impact Of Format Restrictions On Large Language Model Performance." *EMNLP 2024 (Industry Track)*. aclanthology.org/2024.emnlp-industry.91/
- [42] Yuan, H., Zhao, Y., Zhang, L., Luo, W., and Ma, Z. "Quantifying the Impact of Structured Output Format on Large Language Models through Causal Inference." [arXiv:2509.21791](https://arxiv.org/abs/2509.21791), 2025.

- [43] Banerjee, D., Suresh, T., Ugare, S., Misailovic, S., and Singh, G. “CRANE: Reasoning with Constrained LLM Generation.” *ICML 2025*. arxiv.org/abs/2502.09061
- [44] Yao, S., Shinn, N., Razavi, P., and Narasimhan, K. “ τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains.” *arXiv:2406.12045*, 2024.
- [45] Shi, L., Ma, C., Liang, W., Diao, X., Ma, W., and Vosoughi, S. “Judging the Judges: A Systematic Study of Position Bias in LLM-as-a-Judge.” *AACL-IJCNLP 2025*. arxiv.org/abs/2406.07791
- [46] Panickssery, A., Bowman, S. R., and Feng, S. “LLM Evaluators Recognize and Favor Their Own Generations.” *NeurIPS 2024*.
- [47] Khanal, A., Tao, Y., and Zhou, J. “Beyond pass@1: A Reliability Science Framework for Long-Horizon LLM Agents.” *arXiv:2603.29231*, 31 March 2026. *Unreviewed preprint; cited as a caution, not a finding*.
- [52] Barres, V., Dong, H., Ray, S., Si, X., and Narasimhan, K. “ τ^2 -Bench: Evaluating Conversational Agents in a Dual-Control Environment.” *arXiv:2506.07982*, 9 June 2025.
- [53] Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., and Stoica, I. “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena.” *NeurIPS 2023 Datasets and Benchmarks Track*. arxiv.org/abs/2306.05685

Disclosure

The author publishes Work Brain, a personal AI operating system that implements the three-layer model described in §8. The argument in this paper does not depend on that implementation, and the model can be built with ordinary files and a shared drive.

Appendix A — Vendor comparison, gaps shown as gaps

A.1 Overflow behavior when the context window fills

Service	Documented behavior
Claude	Summarizes rather than truncates; conditional on code execution being enabled. Separately, an API primitive named <code>compaction</code> is documented; Anthropic does not state that <code>claude.ai</code> uses it
ChatGPT	Not documented
Gemini	Not documented
M365 Copilot	Not documented (<code>compaction</code> documented only for GitHub Copilot Chat in Visual Studio)
Perplexity	Not documented

A.2 Memory architecture

Microsoft documents Copilot personalization and memory as in preview and subject to change.

Dimension	Claude	ChatGPT	Gemini	M365 Copilot	Perplexity
Automatic	Yes, plus explicit save	Yes	Yes	Yes	Yes

Dimension	Claude	ChatGPT	Gemini	M365 Copilot	Perplexity
Default on	Individual plans yes; Team/Ent no	—	Not stated	Yes	Yes
Scope	Account and per-project	Account; project-only option	Account	Account (Exchange mailbox)	Account; org-owned in Enterprise
User can inspect	Settings, by topic	Memory summary page	Saved Info page	Settings	Settings
Edit granularity	Per topic	Correct the summary	Indirect (delete chats)	Per memory	Per record
Portable	Import/export documented	Not documented	Not documented	Custom instructions only	Not documented
Ownership	User	User	User	User (tenant-discoverable)	Organization (Enterprise)

A.3 Grounding — deterministic or ranked

Service	Documented characterization
Claude Projects	Both, switching automatically by volume; full context until near limit, then RAG, with visual indicator
Claude chat search	Explicitly RAG, surfaced as tool calls
M365 Copilot	Explicitly vector similarity, with query expansion and post-processing retrieval
ChatGPT Projects / GPT knowledge	Not stated
Gemini	Not stated
Perplexity project files	Not stated

A.4 Published capacity limits on durable instruction surfaces

Surface	Limit
Claude organization instructions	3,000 characters
Perplexity project instructions	8,000 characters
Claude Skill description	1,024 characters; ~100 tokens of metadata always loaded per Skill
ChatGPT project files	5 / 25 / 40 by tier; 10 concurrent uploads
Custom GPT knowledge files	20 files, 512 MB each
Gemini prompt files	10 files; 2 GB video, 100 MB other
Copilot declarative agent	4 URLs, 100 SharePoint items, 50 OneDrive files, 5 Teams chats
Claude project knowledge	Not documented
Gem knowledge files	Not documented

A.5 Every published statement of instruction precedence

Vendor	Statement
OpenAI	Model Spec authority hierarchy over message roles: root, system, developer, user, guideline
OpenAI	Project instructions override global custom instructions
Anthropic	Organization instructions take precedence over individual instructions
Anthropic	Ordering among profile, project and Skill instructions — not documented
Google	No instruction hierarchy published
Microsoft	No ordering published across custom instructions, memories, inferred details, agent instructions
Perplexity	No ordering published across project instructions, memories, search history
